Lecture 22 · Part V · Extended notes

Recommender systems: neural, and evaluated honestly

Dataset: MovieLens 1M, binarised

Why these notes exist

Lectures 19–22 sit outside Géron, the course's primary text. For those four lectures *these notes are the primary source*, and they are examined on exactly the same terms as the textbook parts. Everything in the slide deck for this lecture is here, with the arguments written out at length rather than compressed onto a slide; the numbers are the ones the accompanying notebook prints, and every one of them is a measurement on the corpus named above rather than a figure quoted from a paper.

Contents

1 From ratings to rankings

Lecture 21 asked what a user would rate a film. This lecture asks what to show them. They are not the same question, and the second is the one a recommender is actually deployed to answer.

	Lecture 21	**Lecture 22**
Question	what would they rate it?	what should we show?
Output	a number per pair	an ordered list of ten
Evaluated on	ratings they gave	items they had not yet touched
Metric	RMSE	HR@10, NDCG@10 — Lecture 19's
Data	explicit ratings	implicit interactions

Row three changes everything. The evaluation set is no longer a sample of what the user did; it is a claim about what they *would* do, on items they never saw.

1.1 Precisely why RMSE fails here

Lecture 21's RMSE of 0.8587 was computed on held-out ratings — pairs where the user chose to watch the film *and* chose to rate it.

- A recommender operates on the complement of that set: films the user has not chosen.
- RMSE weights every prediction equally, and a recommender shows ten items. Being accurate on the three-star films costs nothing and buys nothing.
- And it is a *pointwise* error, whereas showing ten items is a decision about *order*.

A concrete case

A model that predicts every rating 0.3 stars too low has a poor RMSE and a *perfect* ranking: a constant offset changes no order at all. The two metrics can disagree completely.

1.2 The data, binarised

Call a rating of 4 or 5 an interaction and discard the rest:

Ratings	1,000,209
Positive interactions	575,281
As a fraction of ratings	57.5%
Cells with no interaction	97.4%

Note what the threshold did: a film a user rated 3 is now indistinguishable from one they never saw. We have thrown away the only genuine negatives in the dataset, in order to simulate the situation of a real system — which never had any.

1.3 The metrics, from Lecture 19

HR@10 is the hit rate: is the held-out item in the top ten? For one held-out item this is recall@10, and it is 0 or 1 per user. **NDCG@10** is the same, discounted by position: $1/\log_2(\text{rank} + 1)$ when

hit, 0 otherwise. Both come straight from Lecture 19's derivation with $|R| = 1$. Nothing new is needed, which is the point of having done retrieval first.

HR@10, defined precisely

1. Hold out exactly one interaction per user; call its item g.
2. Score the candidate set for that user; *remove every item they interacted with in the training data*.
3. Take the top ten. HR = 1 if g is among them, else 0.
4. Average over users — macro, one vote each.

Step 2 is where the candidate set is defined, and the candidate set is half of what this lecture measures.

Why already-seen items must be excluded

A small implementation detail with a large effect, and a good example of how a protocol leaks. Leave them in and the popularity baseline recommends the blockbusters every heavy user has already watched — scoring nothing, but occupying the ten slots. Leave them in for the factorisation and it recommends what the user already interacted with, because that is what it was trained to score highly. Which of the two is hurt more depends on the model, so the choice is *not neutral between them*. We exclude them everywhere, and say so.

1.4 The baseline that must be in every table

Recommend the ten most popular items. To everybody. Every time. No personalisation, no model, no parameters — minus the items this user has already interacted with, which is the only thing in it that varies by user.

It is not a straw man

In the reproducibility literature this baseline beats a substantial fraction of published neural recommenders, and by the end of this lecture you will see exactly how that happens. A recommender that does not beat popularity has not been shown to personalise anything.

2 Two towers

2.1 It is Lecture 20, with the query replaced

$$\text{score}(u, i) = E_{\text{user}}(u) \cdot E_{\text{item}}(i)$$

	Lecture 20	**Here**
Left tower	the query text	the user — id, history, context
Right tower	the document	the item — id, genre, text
Precomputed	document vectors	item vectors
At request time	encode the query	encode the user

Identical architecture, identical indexability argument, identical approximate-nearest-neighbour problem. *Retrieval and recommendation converge here*, and that convergence is why Part V is one part rather than two.

2.2 What the towers can hold

Matrix factorisation is the special case where each tower is a *lookup table*: $E_{\text{user}}(u) = p_u$ and $E_{\text{item}}(i) = q_i$, and there the resemblance ends. A tower can instead be a network over *features*: the item's genre, its text, its age; the user's recent history, device, time of day. Which is what buys the cold start — a brand-new item has no q_i to look up, but it does have a genre and a description.

The honest summary of "neural recommenders"

The architecture is a dot product of two learned vectors, exactly as before. What is new is *what the vectors are computed from* — and on datasets with no features beyond ids, there is nothing for the network to compute from. That is a large part of why the reproducibility results came out as they did.

	Lookup table (MF)	**Feature tower**
New item	no vector at all	encode its genre and text
New user	no vector at all	encode context and first actions
Tail item	estimated from very few interactions	shares parameters with similar items

17.9% of films here have fewer than twenty interactions, so row three is most of the catalogue. *This* — not accuracy on the head — is the honest case for a neural recommender, and it is invisible to a metric dominated by popular items.

2.3 Training: the softmax over the catalogue

$$\mathcal{L} = -\log \frac{\exp(s_{ui})}{\sum_{j \in \text{catalogue}} \exp(s_{uj})}$$

The natural objective: the observed item should win against every other item. It is the cross-entropy of Lecture 17 with the catalogue as the classes.

And it is unaffordable. The denominator runs over 3,706 films here, and over tens of millions in a real catalogue — per interaction, per step. So sample the denominator. Which introduces a bias, and the bias has a correction, and the correction is the examinable part.

Sampled softmax and the $\log q$ correction

Draw m negatives from a proposal distribution q and sum over those instead. Popular items are drawn more often, so they are pushed down more often — the sample is not the catalogue, and the gradient knows it. Correct by

$$s'_{uj} = s_{uj} - \log q(j)$$

for each sampled negative. This makes the sampled sum an unbiased estimator of the full one. Without it, the model over-corrects against popular items and systematically under-recommends them.

It is the same correction as in word2vec's sampled objectives, and the same idea as importance weighting: reweight by how likely you were to have drawn the thing you drew.

In-batch negatives, once more

Batch B	Encoder passes	Scores	Negatives per user
8	16	64	7
32	64	1,024	31
128	256	16,384	127
512	1,024	262,144	511
2,048	4,096	4,194,304	2,047

Exactly Lecture 20's table and the same arithmetic — with one difference that matters. In-batch negatives are drawn from the *interaction stream*, so an item's sampling probability is proportional to its popularity, which is precisely the $q(j)$ the correction needs.

Skip the correction and popularity bias is in the gradient

Not in the data, not in the metric — in the gradient itself. It is the cheapest possible bug to introduce and one of the hardest to see afterwards, because the model looks as though it simply learned to dislike popular items.

Where the work really is: not in the architecture, but in the sampling distribution, the correction, the freshness of the item index — item vectors drift as the tower trains, and a stale index is silently wrong, exactly as in Lecture 20 — and the protocol you evaluate under. That is the same list as Lecture 20's, and it is why these two lectures are a pair.

3 The protocol is an experimental variable

3.1 Two decisions nobody reports

To evaluate a recommender you must decide two things, and papers routinely state neither.

1. **Which interaction is held out?** One chosen at random from the user's history, or their last one in time.

2. **Against what is it ranked?** The whole catalogue, or the held-out item against a sample of, say, 100 items the user did not interact with.

Two binary choices, four protocols. We are going to run the same model on the same data under all four.

Commit now

Write down how much you think the four numbers can differ. A few percent? A factor of two? Keep the number; you will want it in three pages.

Stating a protocol, in full

A complete protocol names six things. Most papers name two.

1. How the data was filtered — here, ratings of 4 or 5.
2. What was held out — one interaction per user.
3. How it was chosen — at random, or the most recent.
4. What it is ranked against — the catalogue, or a sample.
5. What was excluded from the candidates — already-seen items.
6. The metric and cutoff — HR@10, NDCG@10.

Change any one and the number changes.

3.2 Decision 1: which interaction is held out

	Random (leave one out)	**Temporal (leave last)**
Held out	one interaction, chosen uniformly	the user's most recent
Training set contains	the user's future	only their past
Matches deployment	no	yes
In the literature	very common	less common

The random split is a leak, and it looks like a convention

Lecture 2 defined a leak: information in the training data that will not be available at prediction time. Check the definition. The model is asked to predict interaction t; it was trained on interactions $t{+}1, t{+}2, \ldots$ by that same user; a deployed system has none of them, because they have not happened.

No column was duplicated, no target was copied, nothing was fitted before the split. The leak is in the *shape of the evaluation*, which is why it survived so long as a default.

3.3 Decision 2: against what is it ranked

	Full catalogue	**Sampled, 100 negatives**
Candidates ranked	3,706	101
Cost	score every item	score 101
Matches deployment	yes — the system ranks the catalogue	no
Why anyone does it	—	it was expensive, once

Sampling was a computational shortcut from a time when scoring a catalogue was expensive. Ranking one held-out item against 100 random others is a far easier task than ranking it against 3,706 — and the 100 are drawn uniformly, so they are mostly obscure films nobody would recommend.

3.4 The experiment

One model: a rank-32 factorisation of the binarised interaction matrix — the simplest two-tower model there is, with both towers lookup tables. One dataset: MovieLens 1M, 575,281 positive interactions. One baseline: the most popular items, minus what the user has already seen. Four protocols: $\{\text{random}, \text{temporal}\} \times \{\text{full}, \text{sampled 100}\}$.

Nothing is tuned per protocol, because tuning per protocol is one of the ways the comparisons in the literature broke. The claim is not that this model is good; it is that *the protocol moves the number more than the model does*, and that argument is cleanest when the model is one you could have written in Lecture 21.

3.5 The table

HR@10	**Full catalogue**	**Sampled 100**
Temporal · factorisation	0.0926	0.6792
Temporal · popularity	0.0404	0.4864
Random · factorisation	0.2132	0.8102
Random · popularity	0.0879	0.6186

The same model on the same data scores 0.0926 and 0.8102 — a factor of 8.8. Nothing about the model changed. Only the two sentences describing how it was scored.

NDCG@10	**Full catalogue**	**Sampled 100**
Temporal · factorisation	0.0464	0.4160
Temporal · popularity	0.0196	0.2656
Random · factorisation	0.1204	0.5630
Random · popularity	0.0462	0.3709

Same shape, a factor of 12.1 across the corners. Worth checking, because "use a better metric" is the natural first response to the HR table — and it does not help. The problem is not the metric; it is what the metric was computed over.

3.6 Take it apart: the split

HR@10, full catalogue	Temporal	Random	Ratio
Factorisation	0.0926	0.2132	×2.30
Popularity	0.0404	0.0879	×2.18

Switching from a temporal to a random split more than doubles both numbers, and does it to the personalised model and the unpersonalised baseline alike.

That last part is the tell

If the gain were the model learning something, it would not lift a popularity list by the same factor. It is *the task getting easier*: predicting a randomly chosen item from a history you have already seen most of.

3.7 Take it apart: the sampling

HR@10, temporal split	Full	Sampled 100	Ratio
Factorisation	0.0926	0.6792	×7.3
Popularity	0.0404	0.4864	×12.0

Sampling is by far the larger effect. HR@10 against 100 random negatives asks "is the right item in the top ten of *101*?" — and ten of 101 is already a tenth of the candidate set. A random ranker scores 0.0990 under this protocol and about 0.0027 against the full catalogue: *the floor moved by a factor of 37*, and every method rides up with it.

Why sampling flatters unequally

The 100 negatives are drawn uniformly, and the catalogue is mostly tail: the top 1% of films hold 8.7% of interactions.

- A uniform sample of 100 therefore contains, in expectation, about one head film — the only items a user might plausibly have chosen instead.
- A popularity model is competing against almost no popular items, so it wins easily.
- A model whose skill is discriminating *among plausible items* has nothing to discriminate.

So sampled metrics are not merely noisy

The distortion depends on what kind of model you have, which is exactly the condition under which a metric can *reverse an ordering*. A bias that were uniform would at least preserve rankings; this one does not. That is the finding the recommender literature had to absorb, and it invalidated a great many published comparisons.

3.8 How large are modelling gains, for comparison?

To read those effects you need a sense of what a real improvement looks like in this field. A new architecture reporting a 5–10% relative gain on HR@10 is a publishable result. Our two protocol

decisions moved HR@10 by 130% and 641% relative, on an unchanged model.

The reproducibility problem, in one comparison

The undocumented choices are an order of magnitude larger than the documented contributions.

3.9 The consequence that ends careers

System, as reported	HR@10
Popularity, random split, sampled 100	0.6186
Factorisation, temporal split, full catalogue	0.0926

A method with *no personalisation whatsoever*, evaluated generously, reports 6.7 times the score of a real factorisation evaluated honestly. Both numbers are correct and both are computed without error. Put them in a table together, as papers do when they quote a baseline from another paper, and the comparison is meaningless — and nothing in either number says so.

3.10 What is stable across protocols

Protocol	Factorisation ÷ popularity
Temporal, full catalogue	×2.29
Random, full catalogue	×2.42
Temporal, sampled 100	×1.40
Random, sampled 100	×1.31

The absolute numbers spanned a factor of 8.8; the ratio to the baseline spans from ×1.31 to ×2.42.

The practical rule

Report the baseline in the same table, under the same protocol. It does not repair the protocol, and it does make your number readable by someone whose protocol differs — which is everyone.

3.11 What the numbers actually license

Under the honest protocol — temporal, full catalogue — the factorisation scores 0.0926 and popularity 0.0404. The model is worth 2.3 times the baseline, and *that* is the real result. Every other number in the table is larger and means less. NDCG@10 tells the same story: 0.0464 against 0.0196.

The honest headline

"A rank-32 factorisation places the user's next film in the top ten 9.3% of the time, against 4.0% for a popularity list, under a temporal split with the full catalogue ranked."
Every clause is load bearing.

Why the honest numbers look so small

9.3% sounds like failure. It is not: the task is to place one specific film in the top ten of 3,706, from a history that ends before it.

Random ranker	0.0027
Popularity	0.0404
Factorisation	0.0926

Against chance the factorisation is 34 times better. Small absolute numbers are what an honest protocol on a hard task looks like — and a field that finds small numbers embarrassing will drift toward protocols that produce large ones. That drift is not fraud, and it does the same damage.

The mean hides the users

HR@10 per user is 0 or 1, so a mean of 0.0926 means precisely this: the held-out film was in the top ten for 9.3% of users and not for the rest. There is no partial credit and no distribution to inspect — the mean *is* the proportion, which makes it honest and makes its confidence interval computable, since it is a proportion over 6,040 users. NDCG@10 does have a spread, and it is rarely reported. "Report the variance" is good advice that means different things for different metrics: for HR the right companion is an interval, for NDCG a distribution.

3.12 So what should you actually do?

1. Rank the full catalogue. It is affordable now; the shortcut is a historical artefact.
2. Split in time, per user, and say so.
3. Include popularity and a tuned classical baseline in the same table.
4. Never quote a number from another paper into your table without re-running it under your protocol.
5. Report coverage and the popularity profile beside the accuracy metric.

All five are cheap. None of them is standard. Every one of them can only make your headline number smaller, which is the whole difficulty.

4 Popularity bias, and what an offline number is worth

4.1 The feedback loop

A recommender is not an observer of its data. It *produces* it. The system recommends popular items because they are the safe bet; users interact with what they are shown, since they cannot interact with what they are not; those interactions become the next training set; and the popular items become more popular still.

Nothing in the objective notices

Every offline metric in this lecture is computed on interactions produced by whatever system was running at the time. A model that reproduces the old system's behaviour scores well, and a model that would have shown something better scores badly — because the evidence for that something was never collected.

It is the pooling problem again

Lecture 19	Lecture 22
unjudged documents counted as irrelevant	unseen items counted as negatives
the pool came from the systems of its day	the log came from the recommender of its day
a novel method is penalised for finding new documents	a novel model is penalised for recommending new items

Three lectures apart, the same structure: the ground truth records what a previous system chose to show, and everything else is scored as wrong. Which is why Part V ends here rather than on an architecture. The architectures converged two lectures ago; the measurement problem did not.

4.2 What to report, beside the metric

- **Catalogue coverage** — what fraction of items are ever in anyone's top ten. A model with a great HR@10 and 2% coverage is a popularity list with extra steps.
- **Popularity profile** — the distribution of recommended items' interaction counts, against the catalogue's.
- **Per-user spread**, not just the mean.
- **The protocol**, in the sentence that states the number.

None of these is hard. They are absent from papers because nothing forces them to be present, and because each of them can only make the headline look worse.

4.3 None of it is the real test

Every number in this lecture is *offline*: computed on a log, about a system that was never run.

Offline	Online
did the model rank the held-out item highly?	did people click, watch, return, cancel?
on a log the old system produced	on traffic the new system produces
a proxy, computable in seconds	the objective, over weeks
no risk	real users see the worse arm

The last row is why offline evaluation exists at all, and the second is why it cannot settle anything: the offline test asks whether you would have reproduced the old system's successes.

The honest position

Use offline evaluation to decide what is worth testing. Use an online test to decide what is worth keeping. Do not use either to make the claim the other one supports.

4.4 One more thing the metric cannot see

A recommender chooses what a person is exposed to, at scale, and the objective it is optimised for is engagement measured by interaction. Content that provokes engages, and engagement is the signal the model is fitted to; the feedback loop then amplifies whatever the objective rewards, including things nobody chose to reward. None of this appears in HR@10, NDCG@10, coverage, or any metric in this lecture.

This course cannot settle what a recommender should optimise. It can insist that you know your objective is a *proxy*, that you can say what it omits, and that "the metric improved" is never by itself an argument that the system got better.

4.5 Sequential recommendation, briefly

Everything so far treated a user as a *set* of interactions. But order carries information: someone who has just watched the first two films of a trilogy is in a *state*, not merely in possession of a history. Model the sequence and predict the next item — which is the language-modelling objective of Lecture 17 with items as tokens — using architectures you already have: a recurrent network (Lecture 16) or self-attention over the history (Lecture 18). And a temporal split becomes *mandatory*, because a random one destroys the very ordering the model is about.

4.6 The convergence, in one table

	Search (Lecture 20)	**Recommendation (Lecture 22)**
Left tower	query	user
Right tower	document	item
Score	dot product	dot product
Negatives	in-batch, hard-mined	in-batch, sampled
Index	ANN over documents	ANN over items
Ground truth	pooled judgements	a log from the old system

Six rows, and only the last two differ in substance. The two halves of Part V are one method with two data-collection problems, and the data-collection problems are the hard part in both.

Exercise 22.1

Drop the sampled negatives from 100 to 20 and re-run. Predict the direction first, then check whether the *ordering* of the two methods survives.

5 Reference

5.1 Five questions for a recommender paper

1. **Full catalogue or sampled?** If sampled, the number is not comparable to anything.
2. **Random split or temporal?** A random split leaks the user's future.
3. **Is popularity in the table**, under the same protocol?
4. **Were the baselines tuned** as carefully as the proposed model, or quoted from elsewhere?
5. **Is there an online result?** If not, the claim is about a log, not about users.

In the reproducibility study, most papers failed at least two.

An exam-style question, worked

"A paper reports HR@10 of 0.8102 for its new model, against 0.0404 for a popularity baseline quoted from an earlier paper. What, if anything, has been established?"

- Nothing. The two numbers are from different protocols, and a quoted baseline is by definition not run under yours.
- Under one protocol, popularity itself reaches 0.6186 — larger than the honest counterpart of the number being celebrated.
- The repair costs nothing: re-run popularity under the paper's own protocol and put it in the same table.

5.2 Where students lose marks

- Comparing HR@10 figures from two papers without checking the protocol. This lecture exists to make that reflex automatic.
- Saying sampled metrics are "noisier but unbiased". They are biased, and non-uniformly across models.
- Omitting the popularity baseline.
- Forgetting to remove already-seen items before ranking.
- Describing a two-tower model as fundamentally different from matrix factorisation. It is the same score with richer encoders.

5.3 Vocabulary

Two-tower	user encoder and item encoder, scored by a dot product
Sampled softmax	a softmax whose denominator is estimated from a sample
$\log q$ correction	subtracting $\log q(j)$ to unbias that estimate
Leave-one-out	hold out one interaction per user, chosen at random
Temporal split	hold out the most recent interaction instead
Sampled metric	rank the held-out item against a sample of negatives
Popularity bias	systematic over-recommendation of already-popular items
Feedback loop	the system shaping the data it will next be trained on

5.4 Scope

Examinable. Why RMSE is the wrong metric for top-N; the two-tower model and its identity with Lecture 20's bi-encoder; sampled softmax and the $\log q$ correction; the four protocols, the direction of each effect, and why sampling can reverse an ordering; the popularity baseline; popularity bias and the feedback loop; the offline/online distinction.

Not examinable — engineering. Specific architectures and libraries, serving, A/B testing machinery.

Beyond the syllabus. Sequential and session-based models in detail, counterfactual and off-policy evaluation, bandits for exploration.

5.5 Reading

- These notes are the primary source.
- Dacrema, Cremonesi and Jannach, *Are We Really Making Much Progress?* — the reproducibility study. Read it before you believe any recommender leaderboard.
- Krichene and Rendle, *On Sampled Metrics for Item Recommendation* — why sampled metrics are not merely noisy.
- Yi et al., *Sampling-Bias-Corrected Neural Modeling* — the $\log q$ correction in a deployed two-tower system.

5.6 Three numbers to leave with

Popularity, honestly measured	0.0404
A rank-32 factorisation, honestly measured	0.0926
Popularity, measured generously	0.6186

Rows one and two are the finding: a real model is worth 2.3 times the trivial baseline. Row three is that *same trivial baseline*, scored under a protocol many papers use, reporting 6.7 times the honest model. If you remember one thing from Part V, make it that row three exists and looks exactly like a result.

5.7 Part V, closed

Lecture 19	a ranking is scored by a question, chosen before the scores
Lecture 20	the first stage sets the ceiling; the old method still won
Lecture 21	the missing entries are the data; do not impute them
Lecture 22	the protocol is part of the result

Four lectures outside the textbook, and not one of them needed a method you had not already met. What they needed was the discipline of Part I applied to problems whose ground truth is itself a measurement.

The Part V checklist

1. Name the metric and the cutoff before seeing scores.
2. Fit the trivial baseline first — corpus order, popularity, the global mean.
3. State the protocol in the same sentence as the number.
4. Ask where the ground truth came from, and what it counts as negative.
5. Report what the metric cannot see, in one line.

5.8 Summary

One line to keep

The protocol is part of the result. A number without it is not a measurement.

A recommender ranks a catalogue for a user, so it is scored by Lecture 19's metrics on items the user had not touched — not by RMSE. The two-tower model is Lecture 20's bi-encoder with users in place of queries, and matrix factorisation is its lookup-table special case. The catalogue softmax is unaffordable, so it is sampled — and a sampled softmax needs the $\log q$ correction or popularity bias enters the gradient. The protocol moved HR@10 from 0.0926 to 0.8102 on one unchanged model, and it lets a popularity list report 6.7 times a real model's honest score. And offline numbers filter ideas; only an online test measures the thing you care about.